COMPUTER PROGRAMMING FOR PETROLEUM ENGINEERS

PROJECT 3- Python Engineering Calculator

INDEX NUMBER : 4072924

```
"""
Engineering Calculator
A menu-driven command-line tool for common petroleum/mechanical engineering
calculations: Reynolds Number, Ideal Gas Volume, Hydrostatic Pressure,
Heat Conduction (Fourier's Law), and Stress from Force.
"""

# ---------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------
GAS_CONSTANT_R = 8.314          # J/(mol.K)
ATMOSPHERIC_PRESSURE = 101325   # Pa
STEEL_YIELD_STRENGTH = 250      # MPa


# ---------------------------------------------------------------------
# Helper functions
# ---------------------------------------------------------------------
def get_positive_float(prompt, allow_zero=False):
    """
    Prompt the user until a valid positive (or non-negative) float is entered.

    Parameters:
        prompt (str): The message shown to the user.
```

```
        allow_zero (bool): If True, 0 is accepted as valid (>= 0).
                           If False, the value must be strictly >
0.

    Returns:
        float: A validated numeric input.
    """
    while True:
        try:
            value = float(input(prompt))
            if allow_zero and value < 0:
                print("  Error: value cannot be negative. Try
again.")
                continue
            if not allow_zero and value <= 0:
                print("  Error: value must be greater than zero. Try
again.")
                continue
            return value
        except ValueError:
            print("  Error: please enter a valid number.")



def sig_fig(value, n=3):
    """
    Format a number to n significant figures as a string.

    Parameters:
        value (float): The number to format.
        n (int): Number of significant figures (default 3).

    Returns:
        str: The formatted number as a string.
    """
```

```
    if value == 0:
        return "0.00"
    from math import log10, floor
    d = n - int(floor(log10(abs(value)))) - 1
    rounded = round(value, d)
    if d <= 0:
        return f"{rounded:.0f}"
    return f"{rounded:.{d}f}"



# ------------------------------------------------------------------
---------
# 1. Reynolds Number
# ------------------------------------------------------------------
---------
def reynolds_number(density, velocity, diameter, viscosity):
    """
    Calculate the Reynolds Number for pipe flow and classify the
flow regime.

    Parameters:
        density (float): Fluid density in kg/m^3.
        velocity (float): Fluid velocity in m/s.
        diameter (float): Pipe internal diameter in m.
        viscosity (float): Dynamic viscosity in Pa.s.

    Returns:
        tuple: (Reynolds number (float), flow regime (str))
    """
    re = (density * velocity * diameter) / viscosity
    if re < 2300:
        regime = "Laminar"
    elif re <= 4000:
        regime = "Transitional"
```

```
    else:
        regime = "Turbulent"
    return re, regime


def run_reynolds_number():
    """Collect inputs and display the Reynolds Number
calculation."""
    print("\n--- Reynolds Number ---")
    try:
        density = get_positive_float("Fluid density (kg/m^3): ")
        velocity = get_positive_float("Velocity (m/s): ")
        diameter = get_positive_float("Pipe diameter (m): ")
        viscosity = get_positive_float("Dynamic viscosity (Pa.s): ")

        re, regime = reynolds_number(density, velocity, diameter,
viscosity)

        print(f"\nReynolds Number (Re): {sig_fig(re)}")
        print(f"Flow Regime: {regime}")
    except Exception as e:
        print(f"  Unexpected error: {e}")


# ---------------------------------------------------------------------
---------
# 2. Ideal Gas Volume
# ---------------------------------------------------------------------
---------
def ideal_gas_volume(pressure, temperature, moles):
    """
    Calculate gas volume using the Ideal Gas Law: V = nRT / P.

    Parameters:
```

```
        pressure (float): Absolute pressure in Pa.
        temperature (float): Absolute temperature in K.
        moles (float): Amount of gas in mol.

    Returns:
        tuple: (volume in m^3 (float), volume in litres (float))
    """
    volume_m3 = (moles * GAS_CONSTANT_R * temperature) / pressure
    volume_l = volume_m3 * 1000
    return volume_m3, volume_l



def run_ideal_gas_volume():
    """Collect inputs and display the Ideal Gas Volume
calculation."""
    print("\n--- Ideal Gas Volume ---")
    try:
        pressure = get_positive_float("Pressure (Pa): ")
        temperature = get_positive_float("Temperature (K): ")
        moles = get_positive_float("Amount of gas (mol): ")

        volume_m3, volume_l = ideal_gas_volume(pressure,
temperature, moles)

        print(f"\nVolume: {sig_fig(volume_m3)} m^3")
        print(f"Volume: {sig_fig(volume_l)} L")
    except Exception as e:
        print(f"  Unexpected error: {e}")



# ----------------------------------------------------------------
---------
# 3. Hydrostatic Pressure
```

```
# ------------------------------------------------------------------
---------

def hydrostatic_pressure(density, depth):

    """

    Calculate hydrostatic (gauge) and absolute pressure at a given
depth.


    Parameters:

        density (float): Fluid density in kg/m^3.

        depth (float): Depth below the fluid surface in m.


    Returns:

        tuple: (gauge pressure in Pa, absolute pressure in Pa,
pressure in bar)

    """

    g = 9.81

    gauge_pressure = density * g * depth

    absolute_pressure = gauge_pressure + ATMOSPHERIC_PRESSURE

    pressure_bar = gauge_pressure / 100000

    return gauge_pressure, absolute_pressure, pressure_bar



def run_hydrostatic_pressure():

    """Collect inputs and display the Hydrostatic Pressure
calculation."""

    print("\n--- Hydrostatic Pressure ---")

    try:

        density = get_positive_float("Fluid density (kg/m^3): ")

        depth = get_positive_float("Depth (m): ", allow_zero=True)


        gauge, absolute, bar = hydrostatic_pressure(density, depth)


        print(f"\nGauge Pressure: {sig_fig(gauge)} Pa")

        print(f"Absolute Pressure: {sig_fig(absolute)} Pa")
```

```
        print(f"Gauge Pressure: {sig_fig(bar)} bar")
    except Exception as e:
        print(f"  Unexpected error: {e}")



# ------------------------------------------------------------------
---------
# 4. Heat Conduction (Fourier's Law)
# ------------------------------------------------------------------
---------
def heat_conduction(k, thickness, t_hot, t_cold, area):
    """
    Calculate heat flux and total heat flow rate using Fourier's
Law.

    Parameters:
        k (float): Thermal conductivity in W/(m.K).
        thickness (float): Wall thickness L in m.
        t_hot (float): Hot side temperature in degC.
        t_cold (float): Cold side temperature in degC.
        area (float): Cross-sectional area in m^2.

    Returns:
        tuple: (heat flux q in W/m^2, total heat flow rate Q in W)
    """
    q = k * (t_hot - t_cold) / thickness
    total_q = q * area
    return q, total_q



def run_heat_conduction():
    """Collect inputs and display the Heat Conduction (Fourier's
Law) calculation."""
    print("\n--- Heat Conduction (Fourier's Law) ---")
```

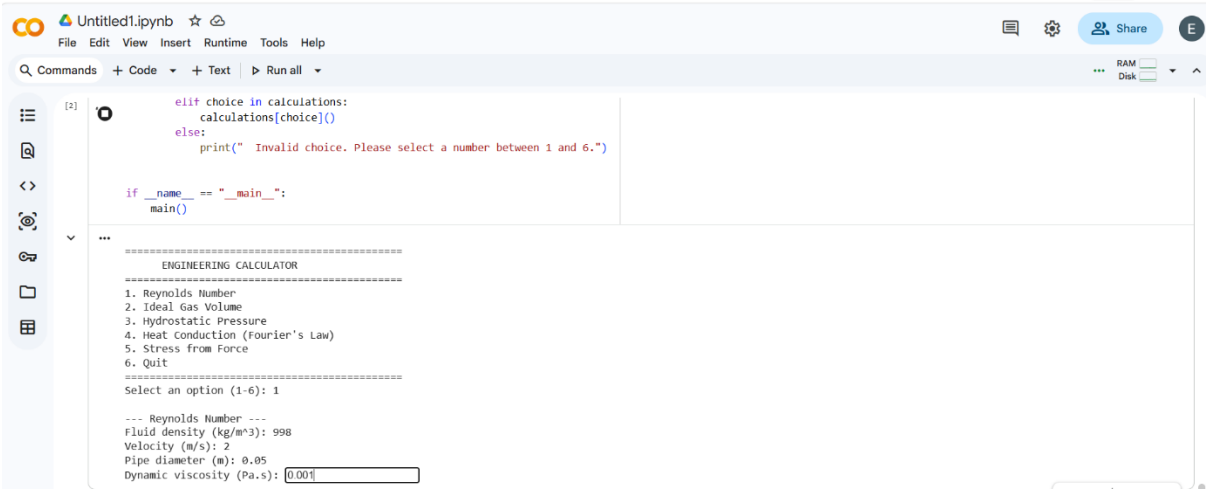
```
    try:
        k = get_positive_float("Thermal conductivity, k (W/m.K): ")
        thickness = get_positive_float("Wall thickness, L (m): ")
        t_hot = float(input("Hot side temperature, TH (degC): "))
        t_cold = float(input("Cold side temperature, TC (degC): "))
        area = get_positive_float("Cross-sectional area (m^2): ")

        if t_hot <= t_cold:
            print("  Warning: TH should be greater than TC for
positive heat flow.")

        q, total_q = heat_conduction(k, thickness, t_hot, t_cold,
area)

        print(f"\nHeat Flux (q): {sig_fig(q)} W/m^2")
        print(f"Total Heat Flow Rate (Q): {sig_fig(total_q)} W")
    except ValueError:
        print("  Error: please enter valid numbers for
temperature.")
    except Exception as e:
        print(f"  Unexpected error: {e}")



# ------------------------------------------------------------------
---------
# 5. Stress from Force
# ------------------------------------------------------------------
---------
def stress_from_force(force, area_mm2):
    """
    Calculate normal stress from an applied force and cross-
sectional area.

    Parameters:
        force (float): Applied force in N.
```

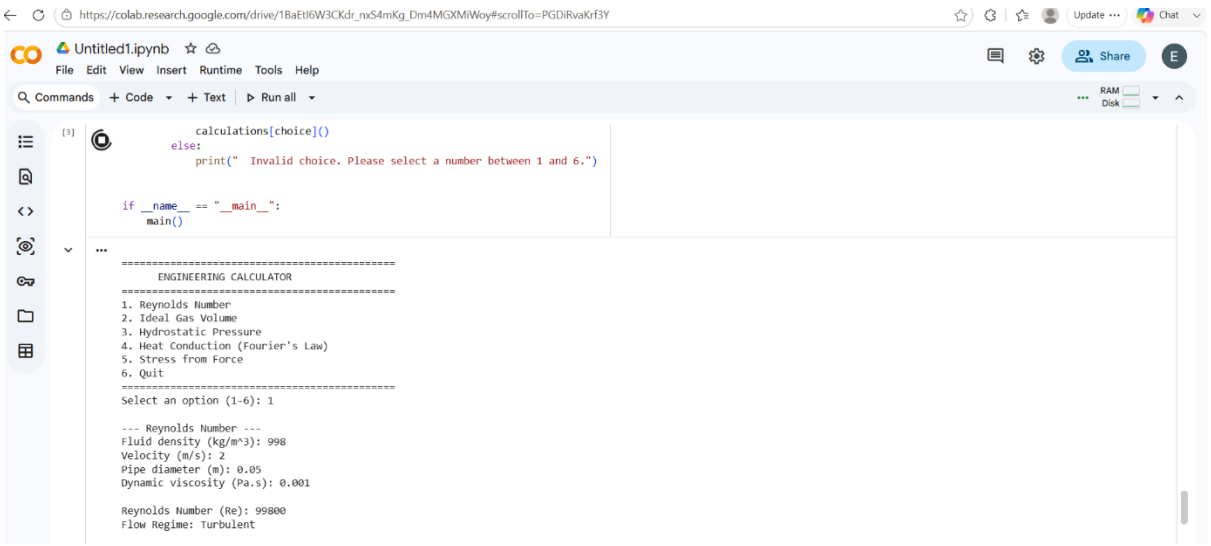
```
        area_mm2 (float): Cross-sectional area in mm^2.


    Returns:
        tuple: (stress in MPa (float), classification relative to
steel yield (str))
    """
    stress_mpa = force / area_mm2
    if stress_mpa > STEEL_YIELD_STRENGTH:
        classification = f"ABOVE steel yield strength
({STEEL_YIELD_STRENGTH} MPa) - risk of yielding"
    else:
        classification = f"below steel yield strength
({STEEL_YIELD_STRENGTH} MPa) - safe"
    return stress_mpa, classification



def run_stress_from_force():
    """Collect inputs and display the Stress from Force
calculation."""
    print("\n--- Stress from Force ---")
    try:
        force = get_positive_float("Applied force (N): ")
        area = get_positive_float("Cross-sectional area (mm^2): ")


        stress_mpa, classification = stress_from_force(force, area)


        print(f"\nNormal Stress: {sig_fig(stress_mpa)} MPa")
        print(f"Classification: {classification}")
    except Exception as e:
        print(f"  Unexpected error: {e}")



# ----------------------------------------------------------------
---------
```

```
# Menu / Main program loop
# ----------------------------------------------------------------------
---------
def display_menu():
    """Print the main menu options."""
    print("\n" + "=" * 45)
    print("      ENGINEERING CALCULATOR")
    print("=" * 45)
    print("1. Reynolds Number")
    print("2. Ideal Gas Volume")
    print("3. Hydrostatic Pressure")
    print("4. Heat Conduction (Fourier's Law)")
    print("5. Stress from Force")
    print("6. Quit")
    print("=" * 45)



def main():
    """Run the main menu loop until the user chooses to quit."""
    calculations = {
        "1": run_reynolds_number,
        "2": run_ideal_gas_volume,
        "3": run_hydrostatic_pressure,
        "4": run_heat_conduction,
        "5": run_stress_from_force,
    }

    while True:
        display_menu()
        choice = input("Select an option (1-6): ").strip()

        if choice == "6":
            print("\nExiting Engineering Calculator. Goodbye!")
```

```
            break
        elif choice in calculations:
            calculations[choice]()
        else:
            print("  Invalid choice. Please select a number between
1 and 6.")


if __name__ == "__main__":
    main()
```

SAMPLE RUN SCREENSHOTS




https://colab.research.google.com/drive/1BaEtl6W3CKdr_nxS4mKg_Dm4MGXMiWoy#scrollTo=PGDiRvaKrf3Y
Update
Chat
Untitled1.ipynb
File Edit View Insert Runtime Tools Help
Share
E
Commands
+ Code
+ Text
Run all
RAM
Disk
[3]
calculations[choice]()
else:
print(" Invalid choice. Please select a number between 1 and 6.")
if __name__ == "__main__":
main()
==========
ENGINEERING CALCULATOR
==========
1. Reynolds Number
2. Ideal Gas Volume
3. Hydrostatic Pressure
4. Heat Conduction (Fourier's Law)
5. Stress from Force
6. Quit
==========
Select an option (1-6): 1
--- Reynolds Number ---
Fluid density (kg/m^3): 998
Velocity (m/s): 2
Pipe diameter (m): 0.05
Dynamic viscosity (Pa.s): 0.001
Reynolds Number (Re): 99800
Flow Regime: Turbulent